Jarosław Sugier

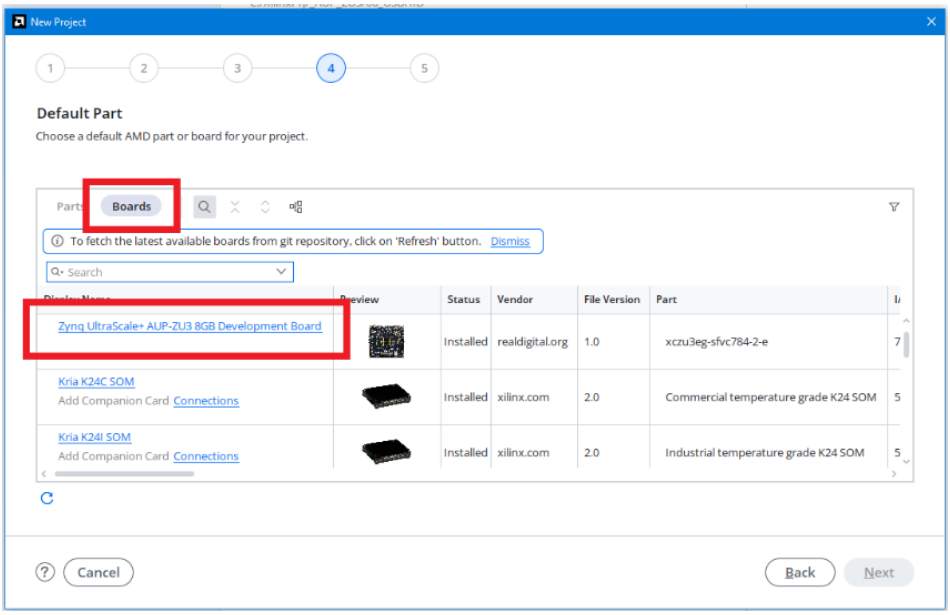
Środowisko Vivado
Instrukcja laboratoryjna

Spis treści

1 Utworzenie projektu 2
2 Pliki źródłowe HDL 2
2.1 Dodawanie nowego pliku 2
2.2 Edycja 4
3 Symulacja 5
3.1 Plik z pobudzeniami (*testbench*) 5
3.2 Definiowanie przebiegów czasowych w języku VHDL 6
3.3 Symulacja behawioralna 6
3.4 Analizowanie przebiegów czasowych 8
3.5 Przebiegi symulacji 8
3.6 Śledzenie sygnałów wewnętrznych 8
3.7 Symulacja czasowa 9
4 Implementacja 10
4.1 Przypisanie wyprowadzeń 10
4.2 Uruchomienie procesu 10
4.3 Konfiguracja układu 12
5 Diagramy blokowe 13
5.1 Tworzenie diagramów 13
5.2 Edycja 14
5.3 Biblioteka IP Vivado 15
5.4 Implementacja 15
6 Sygnały zegarowe 16
6.1 Clocking Wizard w projektach blokowych 16
6.2 Buforowanie w kodzie VHDL 16
6.3 Dzielniki częstotliwości 17

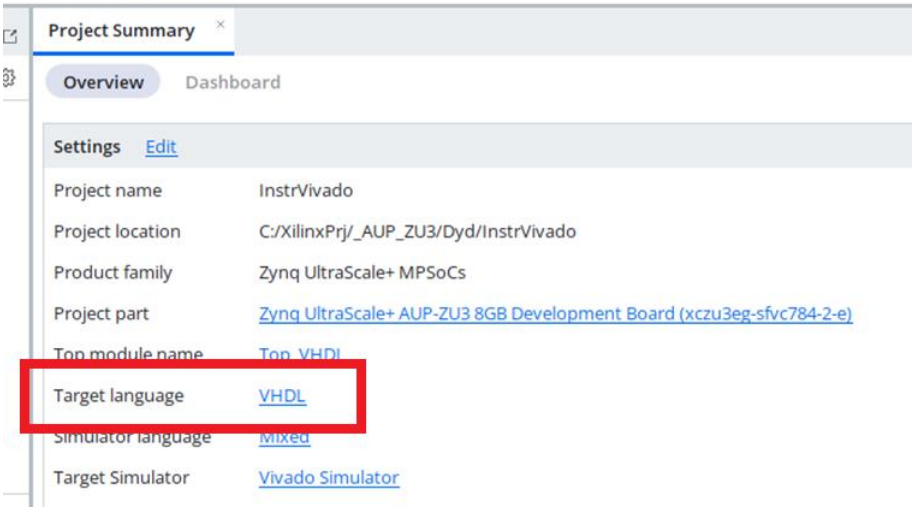
1 Utworzenie projektu

Po uruchomieniu Vivado i wybraniu polecenia `Create project` z ekranu startowego, w kolejnych oknach należy wpisać: nazwę projektu i lokalizację jego folderu, jako typ: `RTL project` oraz wybrać jako platformę płytę AUP-ZU3 8GB:



⇒ Zalecana lokalizacja projektów studenckich w laboratorium: `C:\AMD_Proj`

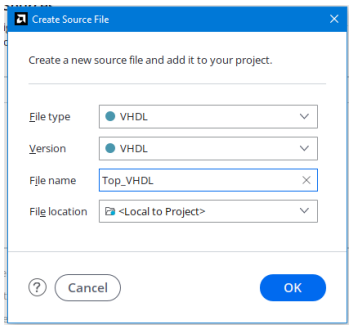
Po utworzeniu projektu w oknie głównym Vivado należy upewnić się, że w podsumowaniu projektu jako język docelowy widnieje VHDL:



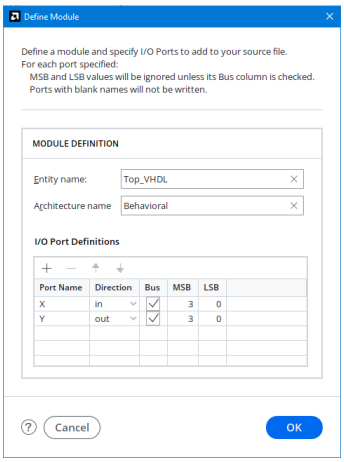
2 Pliki źródłowe HDL

2.1 Dodawanie nowego pliku

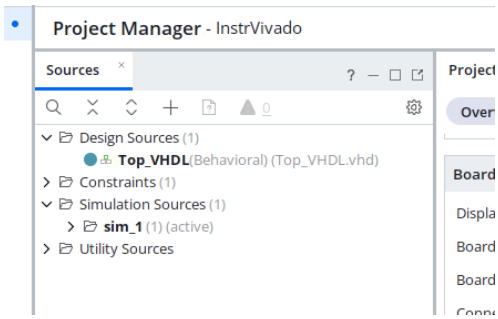
Polecenie `File → Add Sources` (Alt+A) tworzy nowe lub dodaje istniejące pliki jako źródła w projekcie. Tworząc nowy plik do syntezy (`Design source`) należy wybrać język VHDL oraz podać jego nazwę, która będzie także nazwą jednostki VHDL:



W jednym z kolejnych okien trzeba będzie wprowadzić definicję portów:



Poprawność aktualnej hierarchii plików źródłowych należy zawsze kontrolować w oknie `Sources`:



W szczególności, ikona  oznacza element szczytowy hierarchii, który jest punktem startowym implementacji lub symulacji. Punkt ten można ustawić poleceniem `Set as top` w menu kontekstowym pliku.

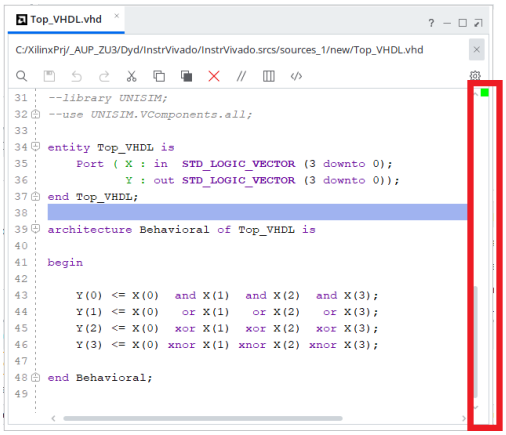
Pliki źródłowe projektu domyślnie umieszczane są w strukturze folderów rozpoczynającej się od `<project>\<project>.srcs`. I tak, nowo tworzone pliki VHDL będą znajdowały się w ścieżce:

`<project>.srcs\sources_1\new\`,

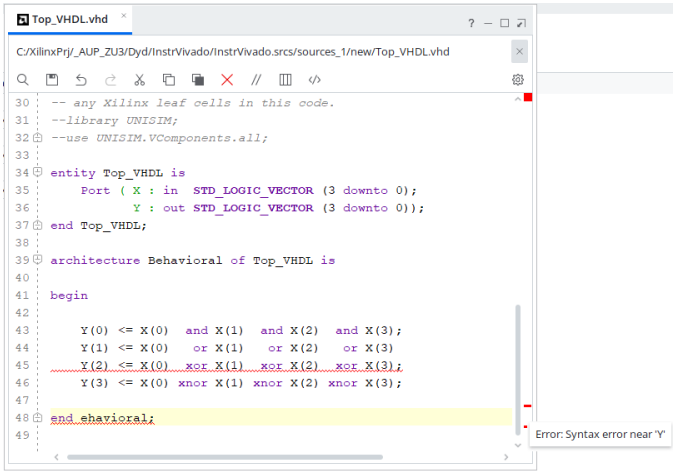
plików do symulacji należy szukać w folderze `<project>.srcs\sim_1`, a ograniczenia projektowe (*User constrains*) – w `<project>.srcs\constr_1`.

2.2 Edycja

Dwukrotne kliknięcie pliku VHDL w oknie hierarchii otwiera edytor kodu. Poprawność składni jest analizowana na bieżąco i sygnalizowana specjalnym pionowym paskiem statusu przy prawej krawędzi okna. Brak błędów sygnalizuje zielony kwadrat na szczycie paska:

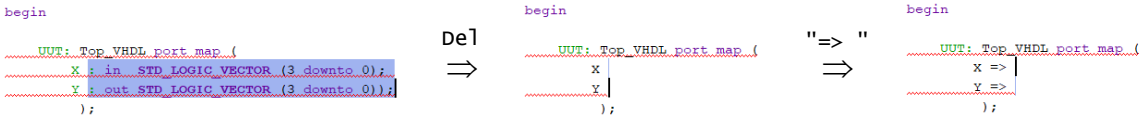


Błędy są oznaczane podkreśleniem na czerwono linii kodu, a dodatkowo na pasku statusu pojawia się czerwona kreska; najechanie na nią kursorem (i kliknięcie) wyświetla komunikat błędu (i ustawia kursor w jego miejscu):



Podczas edycji tekstu można dodawać i usuwać znaki komentarzy na początku linii lub całego zaznaczonego bloku poleceniem `Toggle line comments` (Ctrl+/ lub ikona </>).

Bardzo użyteczny jest tryb kursora blokowego (Ctrl+\ lub ikona ▥, także klawisz Ctrl podczas zaznaczania myszą): po zaznaczeniu bloku można go usunąć (skopiować, wkleić), a kursor obejmujący wiele linii umożliwia wpisywanie w nich tych samych znaków:



3 Symulacja

Symulacja przygotowanego układu wymaga:

1) utworzenia pliku VHDL z wektorami pobudzeń dla wszystkich portów WE (tzw. plik *testbench*),
2) wywołania symulatora ISim, który obliczy oraz przedstawi graficznie odpowiedzi wygenerowane przez układ na portach WY.

3.1 Plik z pobudzeniami (*testbench*)

W środowisku Vivado nie ma narzędzia, które automatycznie utworzy plik *testbench* dla wskazanego pliku źródłowego – należy samemu taki plik dodać do projektu jako nowe źródło VHDL (`Add or create simulation sources`, zob. 2.1) z jednostką VHDL **bez** portów, a następnie wpisać jego treść.

Jak w każdym pliku służącym do symulacji, w części deklaracyjnej architektury należy umieścić (zob. rysunek poniżej):

(a) deklarację komponentu, który ma być symulowany,

(b) definicje sygnałów odpowiadających jego portom (zazwyczaj mają identyczne nazwy – nie będzie tu konfliktu);

a w jej treści:

(c) instancję komponentu (zwyczajowo: o nazwie UUT) oraz przypisanie jego portów do sygnałów,

(d) pobudzenia testowe jako przypisania do portów WE.

Np.:

```
entity tbw_TopVHDL is
end tbw_TopVHDL;

architecture Behavioral of tbw_TopVHDL is

    component Top_VHDL is
        Port ( X : in  STD_LOGIC_VECTOR (3 downto 0);
               Y : out STD_LOGIC_VECTOR (3 downto 0));
    end component;

    signal X : STD_LOGIC_VECTOR (3 downto 0);
    signal Y : STD_LOGIC_VECTOR (3 downto 0);

begin

    UUT: Top_VHDL port map (
        X => X,
        Y => Y
         );

    X <= "0000", "0001" after 100 ns, "0010" after 200 ns, "0011" after 300 ns, "0100" after 400 n

end Behavioral;
```

← definicja jednostki (bez portów)

← (a) deklaracja modułu symulowanego

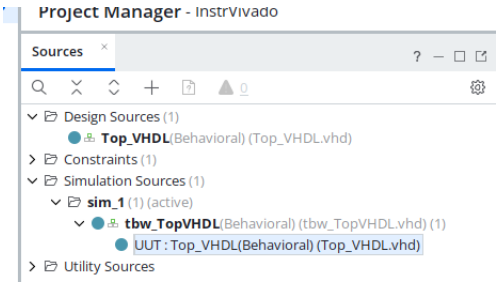
← (b) definicje sygnałów dołączonych do portów

← (c) instancja modułu symulowanego

↑ (a) przypisanie pobudzenia testowego do sygnału WE

⇒ Można skorzystać ze stron automatycznie tworzących *testbench* dla dowolnej jednostki VHDL (np. https://www.doulos.com/knowhow/perl/vhdl-testbench-creation-using-perl/), należy tylko zawsze sprawdzić rezultat i dostosować go do potrzeb projektu.

W oknie plików źródłowych utworzony *testbench* ma być elementem szczytowym w grupie symulacji, a niżej w hierarchii powinna być widoczna znajdująca się w nim instancja modułu symulowanego:



3.2 Definiowanie przebiegów czasowych w języku VHDL

Pobudzenia testowe w pliku *testbench* najprościej opisać instrukcjami przypisań współbieżnych, np.:

```
we1 <= '0', '1' after 100 ns, '0' after 300 ns;
we2 <= '0', '1' after 200 ns, '0' after 400 ns;
```

Dla każdego portu WE należy podać dokładnie jedną instrukcję przypisania. W przykładach powyższych `we1` i `we2` są nazwami portów jednostki, `'0'` oraz `'1'` są przypisywanymi wartościami – stałymi logicznymi, natomiast po słowach kluczowych **`after`** podawane są momenty czasu, w których dane przypisanie ma mieć miejsce.

⇒ Wpisane w takiej instrukcji wartości czasu są momentami wykonania przypisań, a nie długością ich trwania; momenty te muszą tworzyć ciąg ostro rosnący.

⇒ Oznaczenia jednostek czasu w języku VHDL: `fs`, `ps`, `ns`, `us`, `ms`, `sec`.

Sygnałom, które są wektorami (tzn. odpowiadają portom magistralowym), przypisuje się stałe wielobitowe ujęte w znaki "...", np.:

```
BUS4b <= "0000", "0001" after 100 ns, "0011" after 300 ns;
```

Stałe wielobitowe, których długość jest wielokrotnością 4, można zapisywać w notacji heksadecymalnej z przedrostkiem `X`, np.: `"0011" = X"3"`, `"11110011" = X"F3"`, itp. (ale `"011" ≠ X"3"`, bo `X"3" = "0011"`).

Gdy zachodzi potrzeba generacji fali prostokątnej o wypełnieniu 50% i stałym okresie, należy:

a) zainicjalizować wartość początkową sygnału w jego definicji w części deklaracyjnej architektury:

```
SIGNAL Clk : STD_LOGIC := '0';
```

b) w treści architektury zastosować przypisanie sygnału jak w poniższym przykładzie:

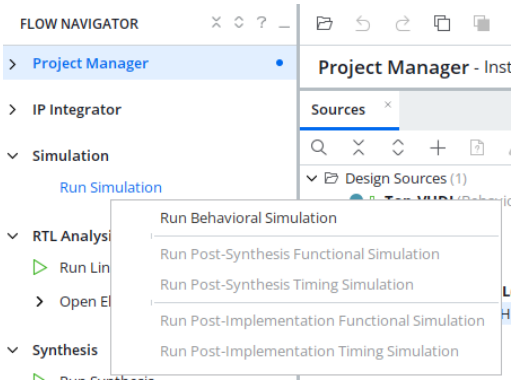
```
Clk <= not Clk after 500 ns;
```

gdzie wartość 500 ns jest zadaną wartość ***półokresu***.

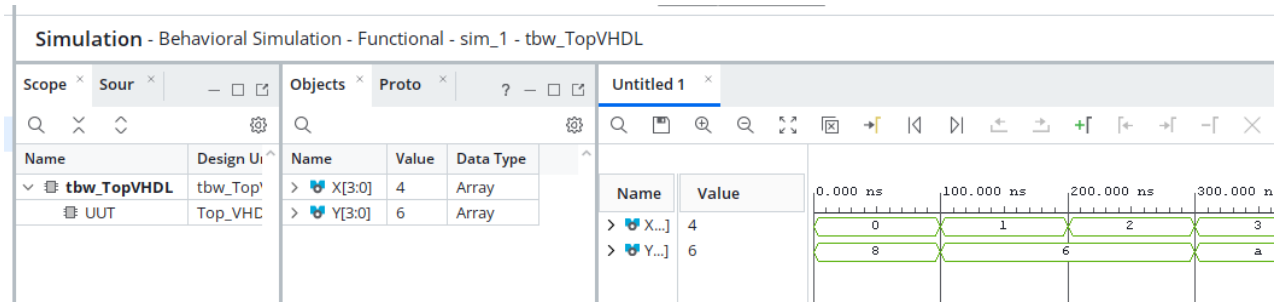
3.3 Symulacja behawioralna

Ten typ symulacji analizuje tylko opis źródłowy VHDL bez jego implementacji sprzętowej w matrycy programowalnej. Nie jest w stanie pokazać opóźnień czasowych wnoszonych przez rzeczywiste

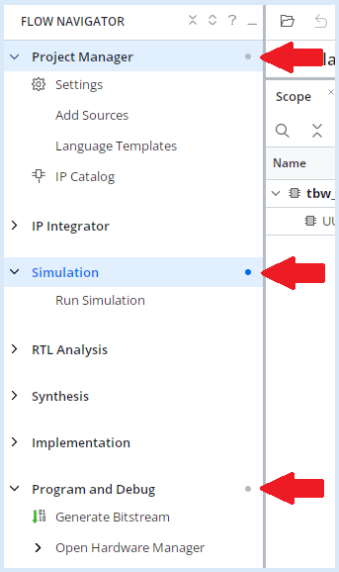
bramki i przerzutniki, ale weryfikuje poprawność pracy projektu na poziomie zero-jedynkowym. Polecenie `Run Behavioral Simulation` znajduje się w kategorii `SIMULATION` okna `FLOW NAVIGATOR`:



Po jego wybraniu uruchamia się kompilacja kodu w plikach *testbench* oraz z jednostką UUT i otwiera się nowe środowisko, w którym – jeśli nie będzie błędów w plikach – m.in. będzie widoczne okno `Waveform` z wynikami symulacji, tj. z przebiegami czasowymi sygnałów:



⇒ Podczas pracy z projektem może być jednocześnie otwartych wiele środowisk, z których tylko jedno jest widoczne, a jego nazwa – `Simulation` na powyższym rysunku – jest wyświetlana na pasku tytułowym obszaru roboczego. Jeśli środowisko jest aktualnie otwarte, jego nazwa w nawigatorze jest oznaczona kropką, np.:

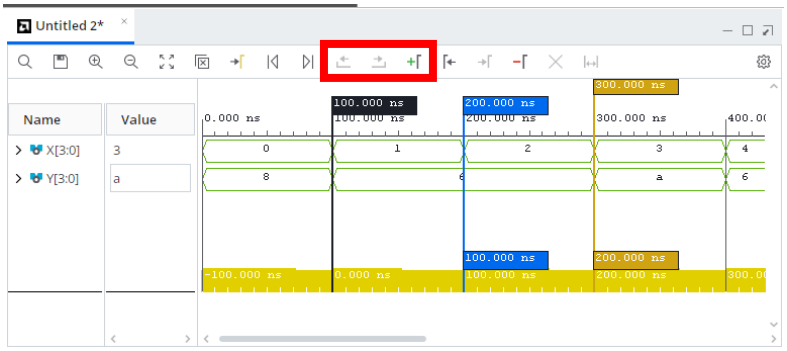


Kliknięcie w nazwę przełącza widok do danego środowiska, a w klawisz x z prawej strony paska tytułowego – jego zamknięcie. Zamknięcie środowiska Project Manager zamyka cały projekt.

3.4 Analizowanie przebiegów czasowych

Okno z przebiegami czasowymi domyślnie jest tworzone z nazwą „Untitled1” i początkowo pokazuje przebiegi wszystkich sygnałów jednostki *testbench*, tj. wszystkich portów UUT.

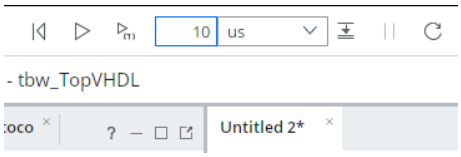
Podstawą analizy jest dobranie odpowiedniej skali wykresu czasowego (`View` → `Zoom In` / `Out` / `Fit`, klawisze). Przy wyznaczaniu opóźnień pomocne są polecenia ustawiające precyzyjnie kursor: po zaznaczeniu dowolnego sygnału polecenia `Previous` / `Next Transition` ( ) przenoszą kursor dokładnie do momentów kolejnych zmian jego wartości. Po ustawieniu znacznika w bieżącej pozycji kursora (`Add Marker`,) można mierzyć odstępy czasowe pomiędzy różnymi punktami wykresu:



Kliknięcie markera ustawia jego moment jako początek skali („czas zero”).

3.5 Przebiegi symulacji

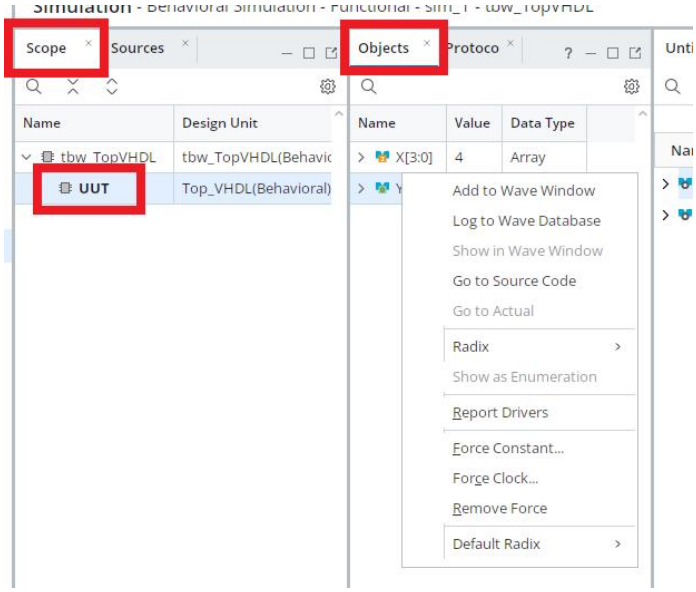
Po pomyślnej kompilacji źródeł ISim wykonuje symulację trwającą domyślnie 1000 ns i zatrzymuje się. Kolejne przebiegi wydłużające wykres można uruchomić wpisując w panelu `Tcl Console` (na dole okna Vivado) instrukcję `run <time>`, np. `run 10 us`. Dodatkowe polecenia dostępne są w pasku menu nad środowiskiem symulacji:



`Restart` (, inicjalizacja symulacji i powrót do chwili 0), `Run for time xx` (10 us) oraz `Relaunch` (, rekompilacja plików źródłowych). Ostatnia komenda jest szczególnie użyteczna, gdy po wprowadzeniu zmian w plikach źródłowych chcemy odświeżyć wynik symulacji bez zamykania i powtórnego uruchamiania całego środowiska ISim.

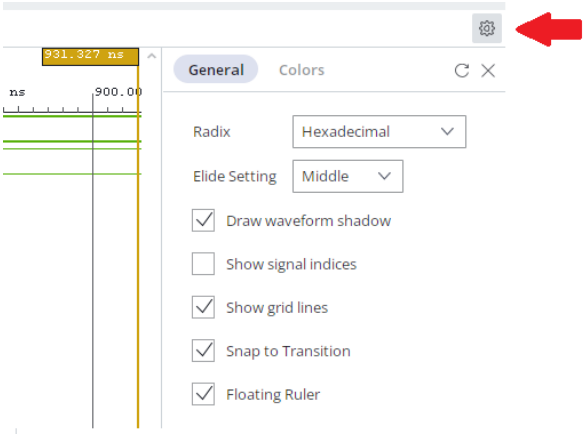
3.6 Śledzenie sygnałów wewnętrznych

Podczas symulacji możliwe jest śledzenie nie tylko portów badanego modułu UUT (które są domyślnie dodawane do okna przebiegów przy jej starcie), ale też wszystkich sygnałów wewnętrznych występujących w hierarchii źródeł. Aby dodać sygnał do okna `Waveform`, po zaznaczeniu obiektu UUT w zakładce `Scope` (lub dowolnego niżej w hierarchii) należy odszukać go w zakładce `Objects` i wybrać polecenie `Add to Wave Window` z jego menu kontekstowego:



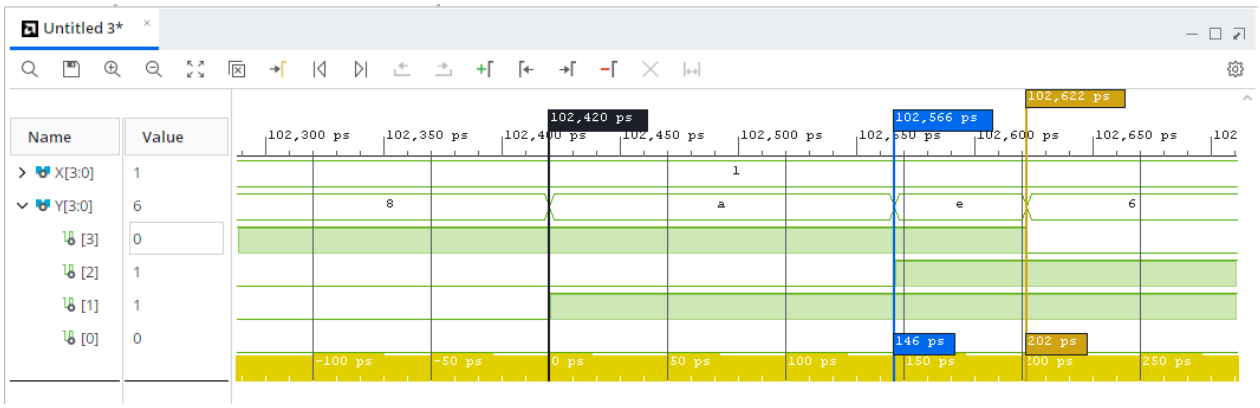
Jeśli przed uruchomieniem symulacji nie zostało włączone logowanie tego sygnału (`Log to Wave Database`, również widoczne powyżej w menu kontekstowym), jego wykres będzie pusty – należy wówczas użyć poleceń `restart` oraz `run <time>`.

Ponadto, inne użyteczne opcje okna `Waveform` są dostępne w jego ustawieniach:



3.7 Symulacja czasowa

Po implementacji projektu (zob. następny rozdział) można uruchomić dokładną symulację czasową finalnego sprzętu wybierając w menu `Run Simulation` plecenie `Run Post-Implementation Timing Simulation`. Widząc opóźnienia propagacji wnoszone przez rzeczywiste elementy matrycy FPGA można analizować ewentualne problemy czasowe w pracy układu:



4 Implementacja

4.1 Przypisanie wyprowadzeń

Przed wygenerowaniem pliku konfigurującego służącego do zaprogramowania płyty konieczne jest przypisanie portów jednostki w szczytowym pliku źródłowym do wyprowadzeń układu Zynq – a więc, do konkretnych urządzeń płyty. Odbywa się to w specjalnej kategorii plików o rozszerzeniach XDC, tzw. *User Constrains*. Należy posługiwać się gotowymi szablonami ZU3.xdc (dla płyty bazowej) oraz ZU3_Exp.xdc (dla płyty rozszerzającej), w których znajdują się - początkowo zakomentowane - definicje przypisań dla wszystkich wyprowadzeń dołączonych do różnych dostępnych urządzeń.

Po pobraniu szablonu ze strony przedmiotu należy dodać go do projektu (`Add or create constrains` jak w 2.1), otworzyć go do edycji klikając w oknie `Hierarchy` i dostosować treść, tzn.
- usunąć komentarze z linii opisujących potrzebne wyprowadzenia
- w miejsce generycznych identyfikatorów (`PL_USER_LED[*]`, `PL_USER_SW[*]`...) wpisać rzeczywiste nazwy portów występujące w szczytowej jednostce VHDL, nie zmieniając pozostałych parametrów.

Plik XDC jest tak naprawdę skryptem języka Tcl zawierającym polecenia `set_property` o składni:

```
set_property <name> <value> <objects>
```

Dla każdego wyprowadzenia (każdego bitu w portach pliku źródłowego) takimi poleceniami muszą być ustawione dwie właściwości:
- `PACKAGE_PIN` (fizyczny numer wyprowadzenia obudowy układu Zynq, np. `D7`, `AA1`, etc)
- `IOSTANDARD` (standard napięciowy, np. `LVDS`, `LVCMOS33`...).

Na przykład dla portu `X(3 downto 0)`, który ma zostać przypisany do przełączników `SW(3:0)` płyty, w pliku XDC należy usunąć komentarze z odpowiednich linii odwołujących się do sygnałów `PL_USER_SW`, zamieniając nich nazwy na `X`:

```
set_property PACKAGE_PIN AB1 [get_ports {X[0]}]
set_property PACKAGE_PIN AF1 [get_ports {X[1]}]
set_property PACKAGE_PIN AE3 [get_ports {X[2]}]
set_property PACKAGE_PIN AC2 [get_ports {X[3]}]
(. . .)
set_property IOSTANDARD LVCMOS12 [get_ports X*]
```

Zazwyczaj wartość `IOSTANDARD` jest ustawiana jednym poleceniem dla całej grupy wyprowadzeń, jak w przykładzie powyżej (`get_ports` z maską nazwy `X*`).

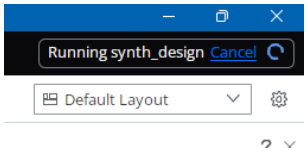
> ⇒ Oprócz przypisania `PACKAGE_PIN` nie można zapomnieć o tej linii w szablonie, która ustawia dla niego `IOSTANDARD`.

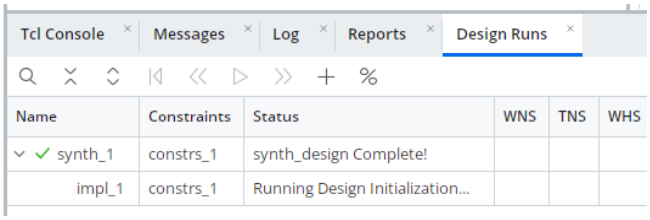
4.2 Uruchomienie procesu

Na implementację projektu składa się ciąg poleceń widocznych w oknie *Flow Navigator*: `Run Synthesis → Run Implementation → Generate Bitstream`. Każde z nich rozpoczyna odpowiedni

przebieg (*Design Run*). Można je uruchamiać sekwencyjnie lub wybrać ostatnie z nich, a wcześniejsze zostaną wywołane automatycznie, jeśli będzie taka potrzeba.

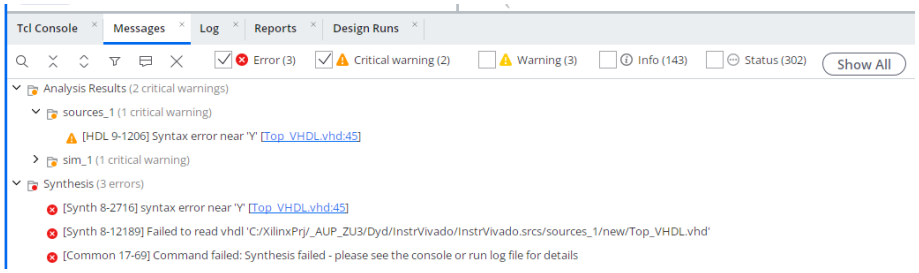
Aktualnie wykonywane przebieg jest wyświetlany w prawym górnym rogu okna głównego (tam też można go przerwać), a zakładka `Design Runs` na dole środowiska pokazuje stan wszystkich przebiegów syntezy i implementacji:





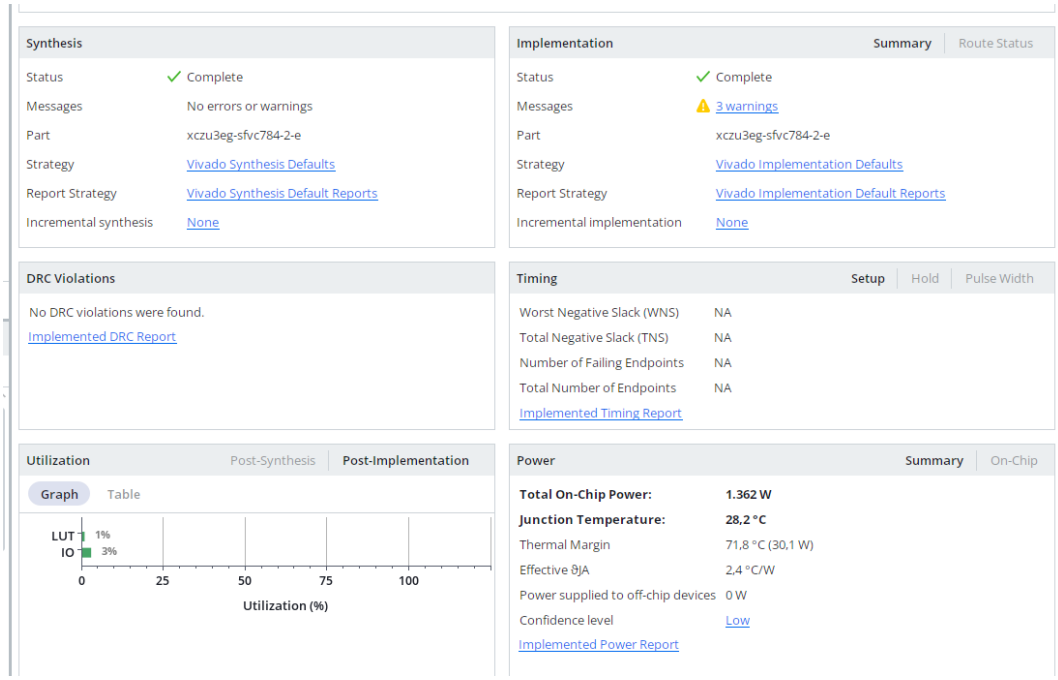
⇒ Do wyzerowania stanu implementacji (np. gdy trzeba usunąć zdezaktualizowane pliki pośrednie i wymusić rozpoczęcie całego procesu od początku) służy polecenie konsoli Tcl `reset_project`.

Komunikaty generowane podczas implementacji widoczne są w zakładce `Messages`:



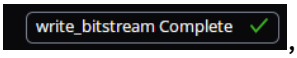
Komunikaty o randze `Status` oraz `Info` zazwyczaj dotyczą pracy środowiska IDE i można wyłączyć ich wyświetlanie. Całą listę komunikatów warto wyczyścić (✕) przed uruchomieniem implementacji. Niektóre nieistotne a szczególnie natarczywe wiadomości można zablokować w ich menu kontekstowym, wybierając `Supress Messages with this ID`.

Podsumowanie rezultatów implementacji pokazuje obszar roboczym okna `Project Summary`:

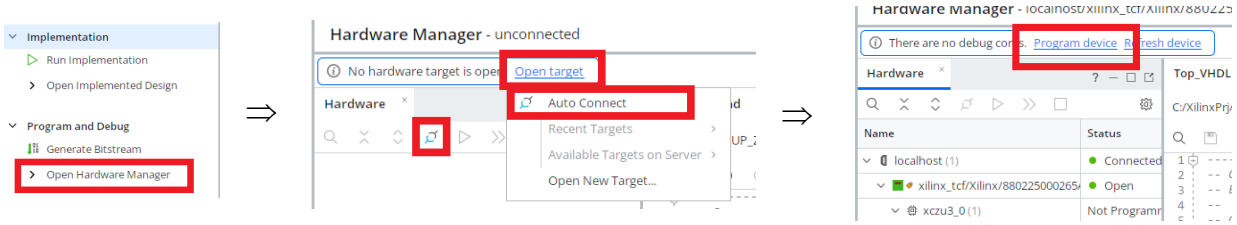


4.3 Konfiguracja układu

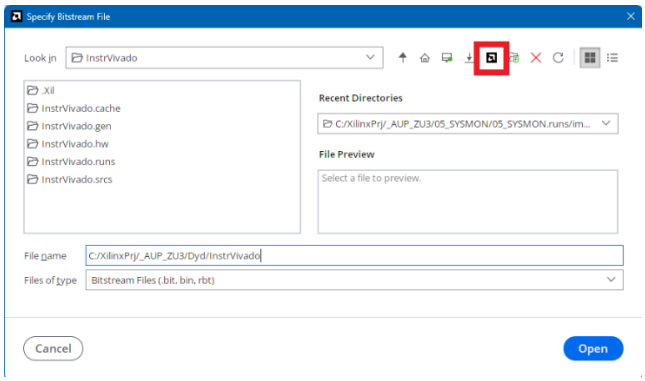
Jeśli po zakończeniu implementacji w prawym górnym rogu okna Vivado widnieje komunikat:



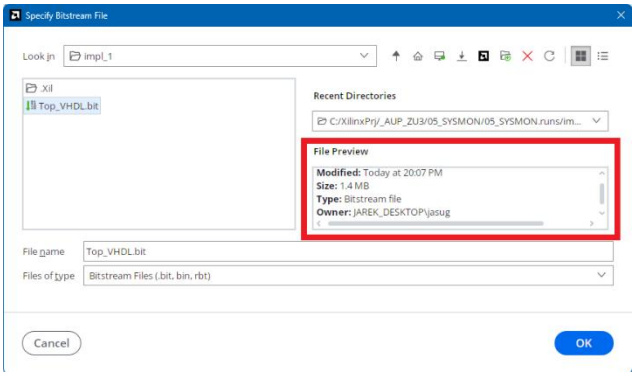
plik konfiguracyjny (*bitstream*) został poprawnie wygenerowany. Ma on rozszerzenie `.BIT` i domyślnie jest tworzony w folderze `<project>.runs\impl_1`. Aby go przesłać należy, po włączeniu zasilania płyty, nawiązać z nią połączenie:



Po kliknięciu `Program device` pojawia się okno `Specify Bitsream File`, w którym należy odszukać i załadować plik BIT. Okno otwiera się w ostatnio używanym folderze, który może być pamiętany z innego projektu – aby mieć pewność, że odszukamy właściwy plik, należy kliknąć ikonę  , która otwiera główny folder projektu bieżącego:



Po wybraniu pliku `.BIT` (z folderu `<project>.runs\impl_1`) warto upewnić się, że ma on poprawną sygnaturę czasową:



Po załadowaniu pliku (`OK`) klawisz `Program` powoduje jego wysłanie do układu – można następnie rozpocząć testy sprzętu.

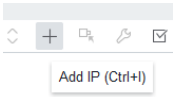
5 Diagramy blokowe

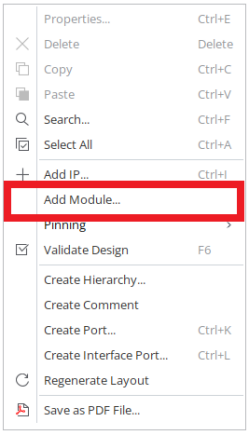
W środowisku Vivado nie ma klasycznego edytora schematów, natomiast jest specjalny edytor diagramów blokowych (*block designs*) przeznaczony do pracy z modułami IP (*Intellectual Property cores*). Moduły IP dodaje się do projektu jako czarne skrzynki z biblioteki systemu Vivado, ale ponieważ w ten sam sposób w diagramach można umieszczać także własne jednostki VHDL (w tym np. gotowe moduły .vhd pobrane ze strony przedmiotu) można - a często warto - korzystać z nich, nawet jeśli nie planujemy używać zaawansowanych rdzeni IP Vivado.

5.1 Tworzenie diagramów

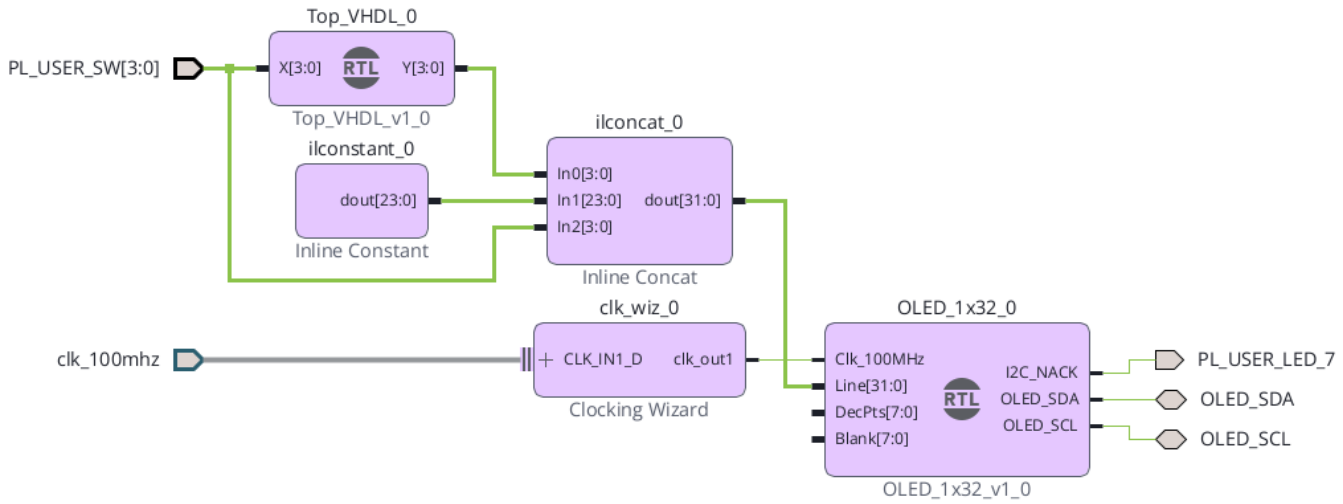
Polecenie `Create Block Design` dostępne jest w kategorii `IP INTEGRATOR` nawigatora; po podaniu nazwy diagramu otwiera się jego edytor graficzny.

Jak już to zauważono, w diagramie można umieszczać dwa rodzaje elementów:
- moduły IP z repozytorium Vivado (ikona `Add IP`, Ctrl+I)
- własne moduły VHDL istniejące w projekcie jako pliki źródłowe (menu kontekstowe `Add Module`...):





Poniższy przykład pokazuje diagram, w którym umieszczono moduły IP z biblioteki Vivado (`Clocking Wizard`, `Constant` oraz `Concat`), moduł OLED_1x32.vhd pobrany ze strony przedmiotu oraz własny moduł źródłowy Top_VHDL:

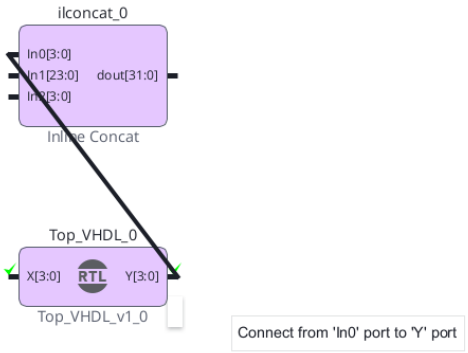


Moduły VHDL mają symbole z etykietą RTL. Po ich zaznaczeniu klawisz F7 otwiera ich kod źródłowy.

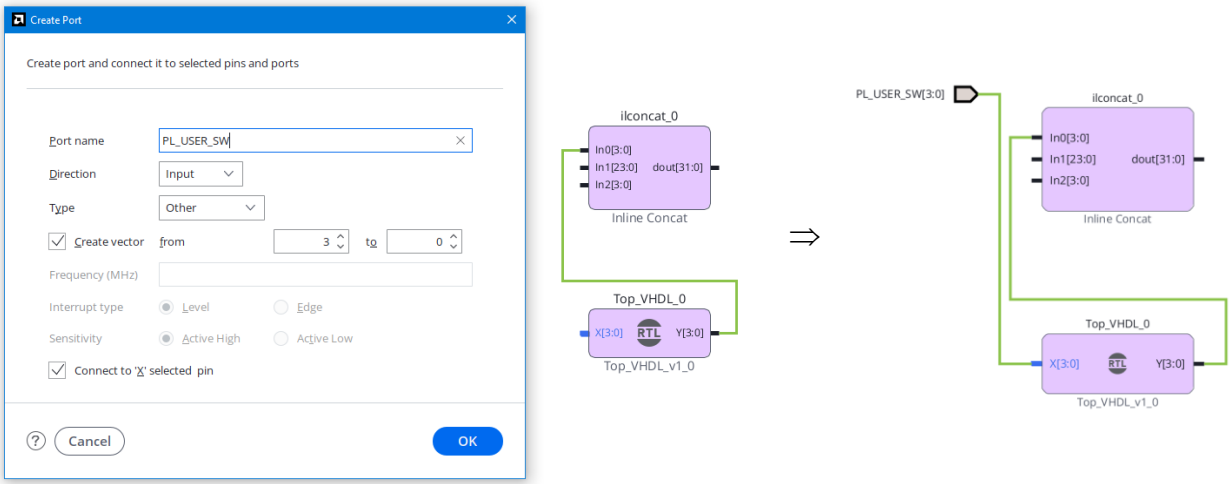
5.2 Edycja

W edytorze graficznym rozmieszczenie modułów oraz trasowanie przebiegu połączeń odbywa się automatycznie i nie należy wymuszać go ręcznie:

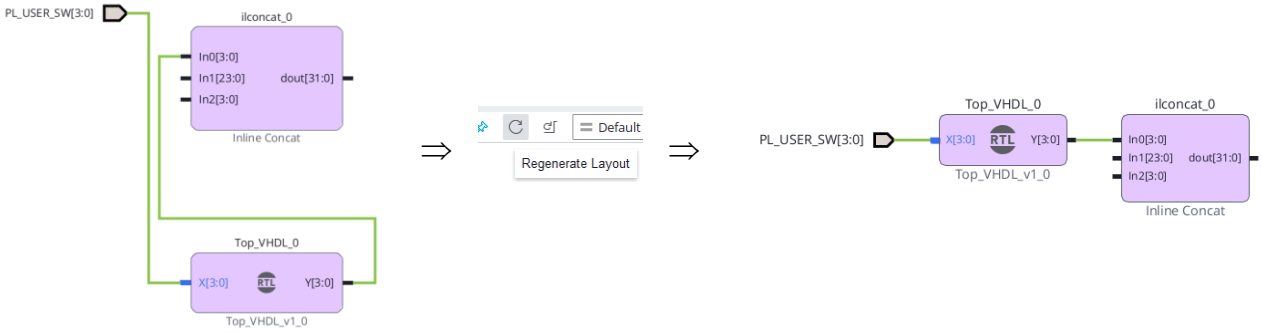
- po dodaniu modułów połącznia między nimi tworzy się przeciągając je myszką z jednego wyprowadzenia na inne:



- aby utworzyć połączenie wyprowadzenia WY modułu z nowym portem I/O należy je zaznaczyć i wybrać polecenie `Create Port…` (Ctrl+K) z menu kontekstowego, wpisując następnie żądane parametry portu (nazwa, ew. indeksowanie):

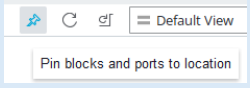


- po utworzeniu połączeń ikona `Regenerate Layout` automatycznie uporządkuje diagram:



Ikona bezpośrednio obok ( ) porządkuje tylko układ połączeń.

⇒ Moduły można przeciągać ręcznie myszką, ale ich rozkład może być regenerowany przy otwarciu diagramu, tzn. pieczołowicie zaprojektowane przez nas rozmieszczenie może zostać stracone. Jeśli szczególnie nam zależy na zachowaniu własnego układu, można użyć ikony `Pin`:



Błędne dołączenie wyprowadzenia do połączenia można usunąć wybierając w jego menu kontekstowym `Disconnect Pin`; nie ma możliwości (ale także nie ma potrzeby przy takich zasadach pracy), by zaznaczyć i usunąć tylko fragment połączenia.

5.3 Biblioteka IP Vivado

Pełną zawartość biblioteki IP można przeglądać uruchamiając `IP Catalog` (polecenie w oknie nawigatora w kategorii `PROJECT MANAGER`). Klikając dwukrotnie symbol modułu IP umieszczonego w diagramie otwieramy jego okno konfiguracji, w którym, oprócz ustawiania parametrów, jest też link do jego dokumentacji (`Documentation` → `Product Webpage` | `Product Guide, etc.`).

⇒ Jeśli jednostka VHDL modułu dodanego do diagramu ma zdefiniowane parametry generyczne, ich wartości także będzie można edytować w oknie konfiguracji. Parametry te muszą być jednak typu całkowitoliczbowego.

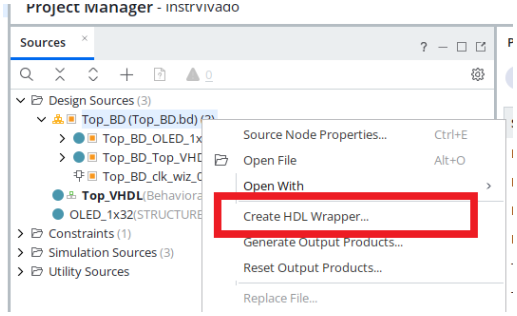
⇒ Edytując parametry w oknie konfiguracji można używać notacji z wykładnikiem, np. 100E6 zamiast 100000000.

W katalogu Vivado znajduje się wiele złożonych modułów IP, ale wśród nich można odszukać także proste, które pozwalają uzyskać w diagramach podstawowe operacje:

- Inline Utility Reduced Logic – wielowejściowa bramka AND | OR | XOR
- Inline Utility Reduced Logic – dla dwóch n-bitowych argumentów generowanie n-bitowego wyniku AND | OR | XOR | NOT (m.in. jedyna metoda uzyskania bramki NOT przy n = 1)
- util_ff – jeden przerzutnik typu D lub wielobitowy rejestr
- Inline Constant – wektor o stałej wartości (m.in. jedyna metoda uzyskania stałych ‘0’ lub ‘1’)
- Inline Concat – sklejanie wielu wektorów WE w jeden
- Inline Slice – podział jednego wektora WE na fragmenty
- Binary Counter - konfigurowalny licznik w kodzie binarnym (opcjonalnie m.in. modulo, dwukierunkowy, etc.)
- Adder/Substracter – sumator / substraktor; po ustawieniu parametru `Latency = 0` pracuje jako układ czysto kombinacyjny

5.4 Implementacja

Diagram blokowy nie może być element szczytowym w implementacji projektu ani użyty w symulacji: konieczne jest wygenerowanie przezroczystego opakowania (*wrapper*) w postaci pliku VHDL, który będzie go reprezentował. Polecenie `Create HDL Wrapper...` jest dostępne w menu kontekstowym, które pojawia się po kliknięciu prawym klawiszem myszy na diagram w oknie źródeł:



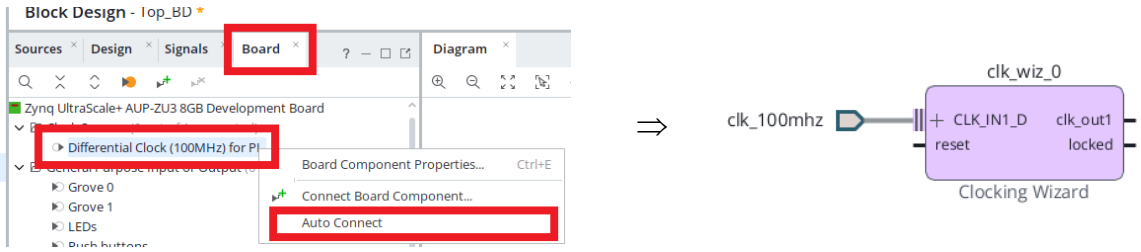
Należy następnie wybrać opcję `Let Vivado manage wrapper and auto-update`. Po wygenerowaniu pliku *wrapper* może on być wybrany jako element szczytowy w implementacji, a zasady przypisywania wyprowadzeń w pliku XDC oraz generowania pliku BIT są identyczna jak opisano to poprzednio dla własnych jednostek VHDL. Tak samo można też umieszczać jego instancję w plikach *testbench* i przeprowadzać symulację.

6 Sygnały zegarowe

Płyta AUP-ZU3 ma tylko jeden generator zegarowy o częstotliwości 100MHz, dołączony do WE różnicowego `CLK_100MHz_P` oraz `CLK_100MHz_N` układu Zynq (zob. plik ZU3.xdc). Taka konfiguracja wymaga odpowiedniego buforowania, które poprawnie wprowadzi taki sygnał do matrycy FPGA.

6.1 Clocking Wizard w projektach blokowych

Jeśli projekt jest opisany diagramem blokowym, obsługę buforowania najlepiej pozostawić modułowi IP `Clocking Wizard`. Podczas edycji diagramu należy w zakładce `Board` zaznaczyć element `Differential Clock` i w jego menu kontekstowym wybrać polecenie `Auto Connect`. Moduł `CLocking Wizard` zostanie automatycznie dodany do diagramu i połączony z odpowiednimi portami:



⇒ Używając ClockingWizard w pliku XDC nie należy opisywać wyprowadzeń `CLK_100MHz_*`; moduł IP dodaje do projektu własny plik *user constrains*, który robi to odpowiednio.

W oknie konfiguracji `Clocking Wizard` (zakładka `Output Clocks`) można zmienić ilość wyjść zegarowych, ich częstotliwości, nazwy, a także wyłączyć opcjonalne WE / WY sterujące (np. widoczne wyżej a często nieużywane `reset` – wymuszenie ponownej inicjalizacji, czy `locked` – sygnalizacja gotowości generatora po osiągnięciu stabilnej pracy).

6.2 Buforowanie w kodzie VHDL

Moduł `Clocking Wizard` może być wstawiony także jako instancja w kodzie VHDL, ale jeśli projekt nie wymaga innej częstotliwości niż 100MHz, można zaoszczędzić miejsce na dysku (oraz przede wszystkim czas implementacji) zastępując go prostszym buforowaniem opisanym wprost w szczytowej jednostce VHDL. Porty zegarowe, jako różnicowe, należy przekazać do bufora o wejściach komplementarnych (`IBUFDS`), a następnie poprzez bufor globalny (`BUFG`) dołączyć do dedykowanej sieci dystrybucji sygnałów zegarowych. W kodzie wymaga to użycia dwóch elementów z biblioteki UNISIM, jak pokazuje poniższy przykład:

```
library UNISIM;
use UNISIM.VComponents.all;

entity Top_VHDL is
```

```
    Port(
       CLK_100MHz_P : in STD_LOGIC;
       CLK_100MHz_N : in STD_LOGIC;
       ( . . . )
end Top_VHDL;

architecture Behavioral of Top_VHDL is
    ( . . . )
    signal Clk_Buf   : std_logic; -- internal connection IBUFDS -> BUFG
    signal Clk_100MHz: std_logic; -- THE global clock for the design
    ( . . . )
begin

    inst_IBUFDS : IBUFDS port map(
        I  => CLK_100MHz_P,  -- 1-bit in:  Diff_p buffer input
        IB => CLK_100MHz_N,  -- 1-bit in:  Diff_n buffer input
        O  => Clk_Buf        -- 1-bit out: Buffer output
        );

    inst_BUFG : BUFG port map(
        I => Clk_Buf,        -- 1-bit in:  Clock input.
        O => Clk_100MHz      -- 1-bit out: Clock output.
        );

    ( . . . )
```

W odróżnieniu od użycia IP `CLockingWizard`, przy takim rozwiązaniu plik XDC musi zawierać linie opisujące wyprowadzenia `CLK_100MHz_*`, tzn. w szablonie ZU3.xdc muszą one być odkomentowane.

6.3 Dzielniki częstotliwości

Używając IP `Clocking Wizard` można syntezować inne częstotliwości niż wejściowe 100MHz, jednak nie mniejsze niż ok. 5MHz. Jeśli potrzebne są wartości poniżej tego limitu, należy stworzyć własny dzielnik oparty o prosty licznik binarny, a jego wyjście dołączyć do globalnej sieci zegarowej przez instancję `BUFG` jak wyżej.

W bibliotece modułów na stronie ePortalu znajduje się plik ClockDivider.vhd, który zawiera pełny kod takiego dzielnika . Zdefiniowana jednostka ma dwa parametry generyczne:

```
entity ClockDivider is
    Generic(
        IN_FREQ_HZ  : POSITIVE := 100E6; -- input freq.(typical 100MHz)
        OUT_FREQ_HZ : POSITIVE := 1      -- requested output frequency
        );
    Port (
        Clk_In  : in  std_logic;
        Clk_Out : out std_logic
        );
end entity;
```

Ze względu na ograniczenia edytora diagramów parametr `OUT_FREQ_HZ` jest typu `POSITIVE`, tak więc minimalna częstotliwość WY jest ograniczona do 1Hz, ale – jeśli moduł będzie użyty jako instancja VHDL and nie w diagramie – kod można łatwo zmodyfikować tak, by parametr ten był typu `REAL`.